

Tema 5

Evaluación de Sistemas Informáticos. Referenciación

Objetivos

Entender la problemática inherente al diseño de un índice de rendimiento cualquiera.

Interpretar los índices clásicos de rendimiento usados en el ámbito de los Sistemas Informáticos.

- Entender el concepto de benchmark y sus distintos tipos.
- Conocer ejemplos reales de benchmarks.

Conocer diferentes estrategias de análisis para hacer comparaciones de rendimiento, así como las condiciones para hacer una comparación de rendimiento lo más ecuánime posible.

Bibliografía



Evaluación y modelado del rendimiento de los sistemas informáticos. Xavier Molero, C. Juiz, M. Rodeño. Pearson Educación, 2004. Capítulo 3.



Measuring computer performance: a practitioner's guide. David J. Lilja, Cambridge University Press, 2000. Capítulo 7.



Performance Tuning for Linux Servers. Sandra K. Johnson et al, IBM Press, 2005. Capítulo 6.



The Standard Performance Evaluation Corporation (SPEC), <http://www.spec.org>.



The Transaction Processing Performance Council (TPC), <http://www.tpc.org>.

Contenidos

Introducción.

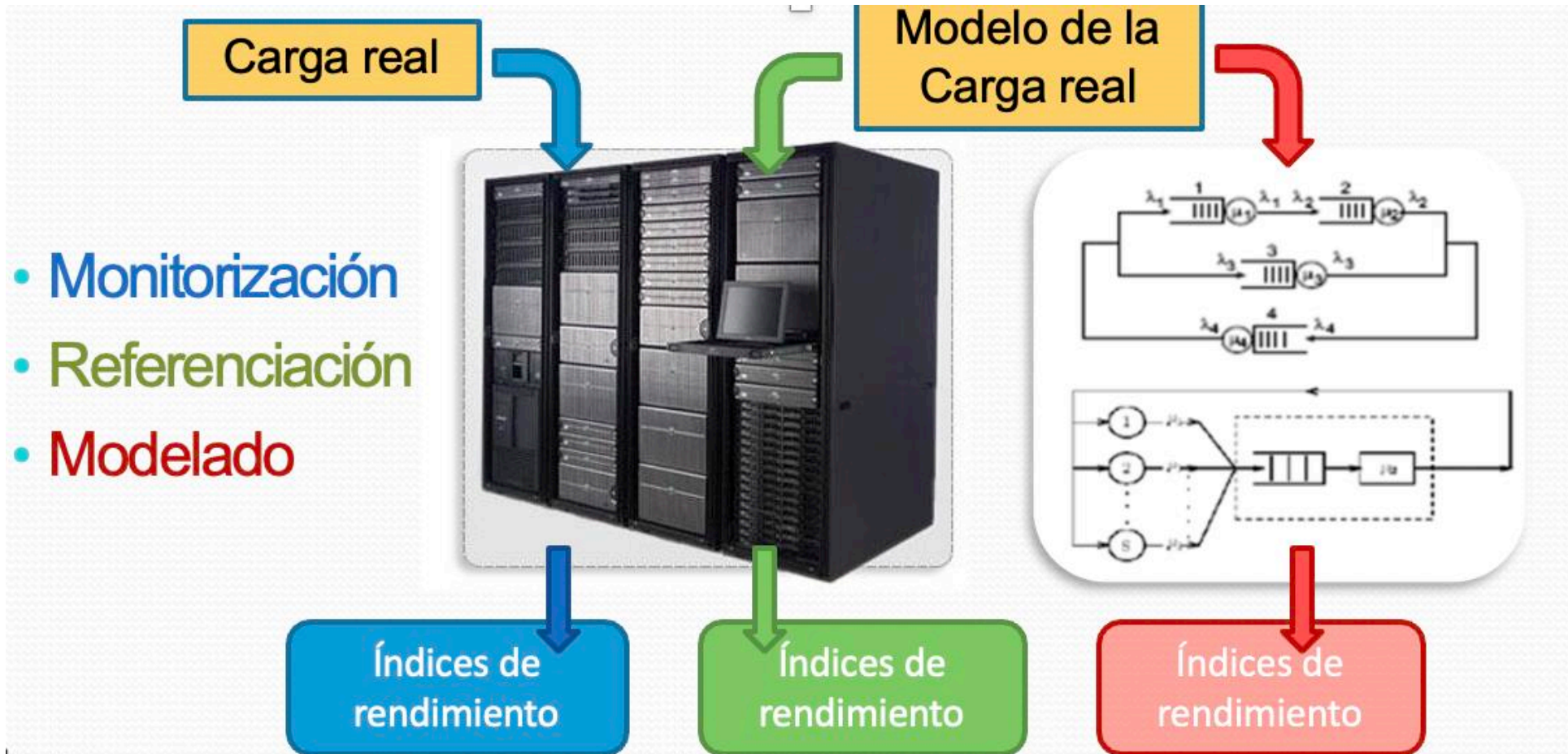
Índices clásicos de rendimiento

Benchmarking

Análisis estadístico de los resultados de un benchmark

Introducción

¿Cómo podemos **evaluar**? (medir, predecir o comparar el rendimiento)



Características de un buen índice de rendimiento de un sistema informático



- **Representatividad y fiabilidad:** Si un sistema A siempre presenta un índice de rendimiento mejor que el sistema B, es porque siempre el rendimiento real de A es mejor que el de B.
- **Linealidad:** Si el índice de rendimiento aumenta, el rendimiento real del sistema debe aumentar en la misma proporción.
- **Repetibilidad:** Siempre que se mida el índice en las mismas condiciones, el valor de éste debe ser el mismo.
- **Consistencia y facilidad de medición:** El índice debe ser fácil de medir y la forma de medirlo debe ser la misma para cualquier sistema.

Tiempo de ejecución, frecuencia de reloj y ciclos por instrucción

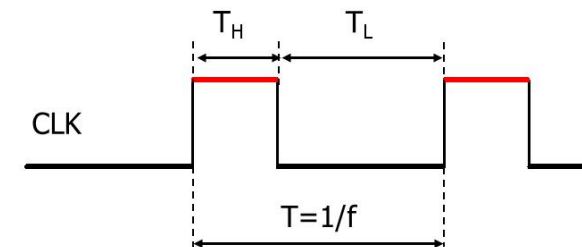
El tiempo de ejecución es el mejor índice de rendimiento a priori pero depende del programa o programas que se ejecuten. ¿Existen otros índices posibles para expresar el rendimiento del sistema? Históricamente se han usado f_{RELOJ} , CPI, MIPS y MFLOPS:

$$T_{\text{EJEC}} = NI \times CPI \times T_{\text{REL-J}} = \frac{NI \times CPI}{f_{\text{REL-J}}}$$

NI = Número de instrucciones del programa/programas a ejecutar.

f_{RELOJ} = frecuencia de reloj.

CPI = N° medio de ciclos por instrucción.



Desventajas:

Ni la frecuencia de reloj ni los ciclos por instrucción son representativos del rendimiento de un sistema. Es posible encontrar ejemplos de sistemas con f_{RELOJ} (o CPI) peores que otros pero con mejores prestaciones.

MIPS

MIPS (*million of instructions per second*)

Se denominan MIPS nativos

$$MIPS = \frac{\text{Instrucciones ejecutadas}}{\text{Tiempo de ejecución} \times 10^6} = \frac{\text{Frecuencia de reloj de la CPU}}{CPI \times 10^6}$$

Depende del juego de instrucciones y los MIPS medidos varían entre programas en el mismo computador

Meaningless indicator of processor speed

MIPS relativos: referidos a una máquina de referencia (proceso de normalización)

$$MIPS_{\text{relativos}} = \left(\frac{\text{Tiempo de referencia}}{\text{Tiempo de ejecución}} \right) \times MIPS_{\text{referencia}}$$

MFLOPS

MFLOPS (*million of floating-point operations per second*)

Basado en operaciones y no en instrucciones

El tiempo de ejecución de la fórmula es el del programa, incluyendo el tiempo consumido por las instrucciones de enteros

$$MFLOPS = \frac{\text{Operaciones de coma flotante ejecutadas}}{\text{Tiempo de ejecución} \times 10^6}$$

Problema: El formato de los números en coma flotante puede variar de una arquitectura a otra y, por tanto, tener diferente precisión.

MFLOPS normalizados

Consideran la complejidad de las operaciones en coma flotante

Suma, resta, multiplicación, comparación, negación: poco costosas

División, raíz cuadrada: costosas

Trigonométricas: muy costosas

Ejemplo de normalización de operaciones en coma flotante

- ADD, SUB, COMPARE, MULT $\Rightarrow$ 1 operación normalizada
- DIVIDE, SQRT $\Rightarrow$ 4 operaciones normalizadas
- EXP, SIN, ATAN, ... $\Rightarrow$ 8 operaciones normalizadas

Cálculo de los MFLOPS de un programa

Programa Spice: el computador DECStation 3100 tarda en 94 segundos en ejecutarlo

Contiene 109.970.178 operaciones en coma flotante de las cuales:

15.682.333 son divisiones (DIVD)

El resto tiene una complejidad similar a la de la suma

$$MFLOPS_{nativos} = \frac{109.970.178}{94 \times 10^6} = 1,2$$

$$MFLOPS_{normalizados} = \frac{94.287.845 + 15.682.333 \times 4}{94 \times 10^6} = 1,7$$

Benchmarking

La carga real

Difícil de utilizar en la evaluación de sistemas

Varía a lo largo del tiempo.

Resulta complicado reproducirla.

Interacciona con el sistema informático.



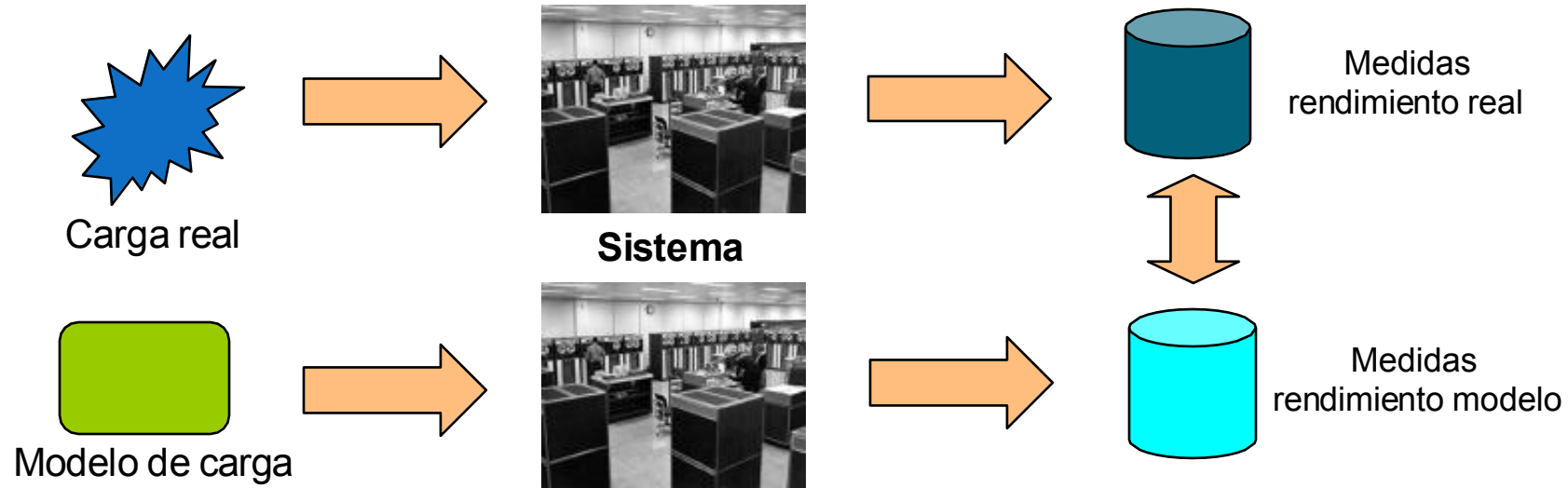
Es más conveniente utilizar un **modelo** de la carga real como carga de prueba (test workload) para hacer comparaciones.

Representatividad del modelo de carga

Los modelos de carga son aproximaciones que representan una abstracción de la carga que recibe un sistema informático. La representación de la carga:

Debe de ser lo más representativa posible de la carga real.

Debe ser lo más simple/compacto posible (tiempos de simulación y espacio en memoria razonables).



Principales estrategias para obtener modelos de carga

Ajustar un modelo paramétrico “**personalizado**” a partir de la monitorización del sistema ante la carga real (*caracterización de la carga*).



Usar programas de prueba que usen un modelo **genérico** de carga lo más similar posible al que se quiere reproducir (*referenciación o benchmarking*).



Caracterización de la carga

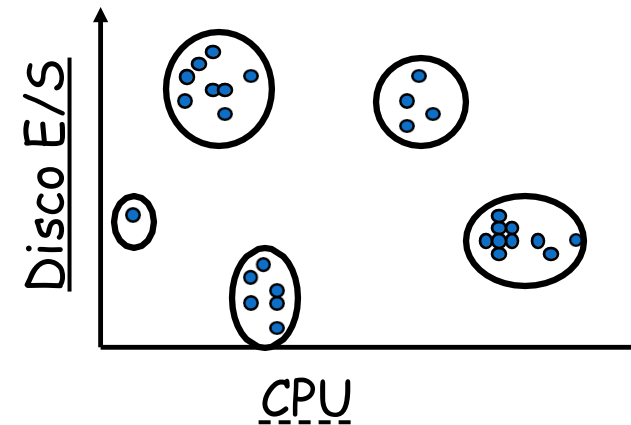
Usualmente la caracterización de la carga de un sistema se realiza siguiendo los siguientes pasos:

- Identificación de los componentes básicos de la carga (CPU, memoria, discos, red, etc.)

- Elección de los parámetros característicos de dichos componentes (frecuencia de llegada de trabajos, lecturas/escrituras que hay que hacer en cada disco, lecturas/escrituras a memoria, necesidades de procesamiento de cada CPU, etc.)

- Recolección de datos (usando monitores de actividad).

- Análisis y clasificación de los datos (medias, histogramas, agrupamiento o *clustering*, etc.)



Referenciación (Benchmarking)

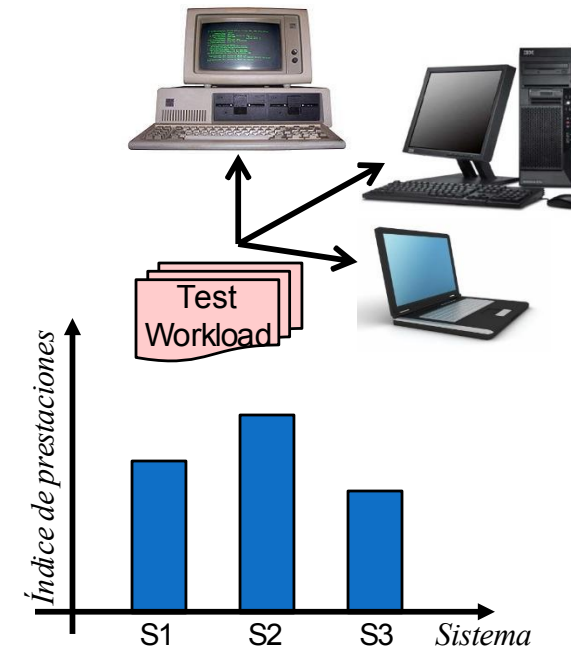
Técnica usada en la comparación del rendimiento de diferentes sistemas informáticos.

Todos los sistemas se han de someter a la misma carga, por lo que ésta ha de ser suficientemente genérica.

Un benchmark (*benchmark program*) es un programa o un conjunto de programas diseñados con el fin de comparar alguna característica del rendimiento entre equipos informáticos. Características principales:

La **carga de prueba** (*test workload*) específica con la que estresa el sistema evaluado.

El **conjunto de reglas** que se deben seguir para la correcta ejecución y validación de los resultados.



Prestaciones = f (carga de prueba)

Tipos de programas de benchmark: según la estrategia de medida

Programas que miden el tiempo necesario para ejecutar una cantidad pre-establecida de tareas.

La mayoría de benchmarks.

Programas que miden la cantidad de tareas ejecutadas para un tiempo de cómputo pre-establecido.

SLALOM: Mide la precisión de la solución de un determinado problema que se puede alcanzar en 1 minuto de ejecución.

Programas que permiten variar tanto la cantidad de tareas como el tiempo de cómputo para adaptarlos a cada sistema.

HINT: Calcula los límites inferior y superior de una integral hasta que el sistema se quede sin recursos. Medida de rendimiento: QUIPS (*quality improvements per second*).

Tipos de programas de benchmark: según la generalidad del test

Microbenchmarks o benchmarks para **componentes**: estresan componentes o agrupaciones de componentes concretos del sistema: procesador, caché, memoria, discos, red, procesador+caché, memoria virtual, etc.

Benchmarks de sistema **completo**: carga compuesta por un conjunto de aplicaciones, normalmente comerciales, habitualmente utilizadas en algún área, p.ej. e-comercio, servidores web, servidores de ficheros, servidores de bases de datos, sistemas de ayuda a la decisión, paquetes ofimáticos + correo electrónico + navegación, etc.

Benchmarks desarrollados **específicamente** por el usuario: se obtiene un modelo de carga que representa la carga habitual (o futura) de un equipo concreto con el fin de comparar el rendimiento de este ante diferentes configuraciones.

Ejemplos de microbenchmarks

Whetstone (1976)

Mide el rendimiento de las operaciones en coma flotante por medio de pequeñas aplicaciones científicas que usan sumas, multiplicaciones y funciones trigonométricas.

Linpack (1983)

Mide el rendimiento de las operaciones en coma flotante a través de un algoritmo para resolver un sistema denso de ecuaciones lineales. El benchmark incorpora una rutina para comprobar que la solución a la que se llega es la correcta con un grado de precisión prefijado.

Dhrystone (1984)

Mide el rendimiento de operaciones con enteros, esencialmente por medio de operaciones de copia y comparación de cadenas de caracteres.

Ejemplos de microbenchmarks (II)

Stream: para medir el ancho de banda de la memoria

<http://www.streambench.org/>.

IOzone: rendimiento del sistema de ficheros (lecturas y escrituras a/desde el disco duro). <http://www.iozone.org/>. Igualmente HD Tune (Windows, <http://www.hdtune.com/>), o el comando 'hdparm -tT' (Linux).

Netperf: rendimiento TCP y UDP (Linux y Windows). Basado en una arquitectura cliente-servidor, se usa en combinación con otro programa (netserver) que debe estar instalado en el servidor. <http://www.netperf.org/netperf/>

También existen aplicaciones que incorporan varios **paquetes de microbenchmarks** para poder realizar diversos tests de forma cómoda:

LMbench (Unix, <http://lmbench.sourceforge.net/>),

AIDA64 (Windows, <http://www.aida64.com/>),

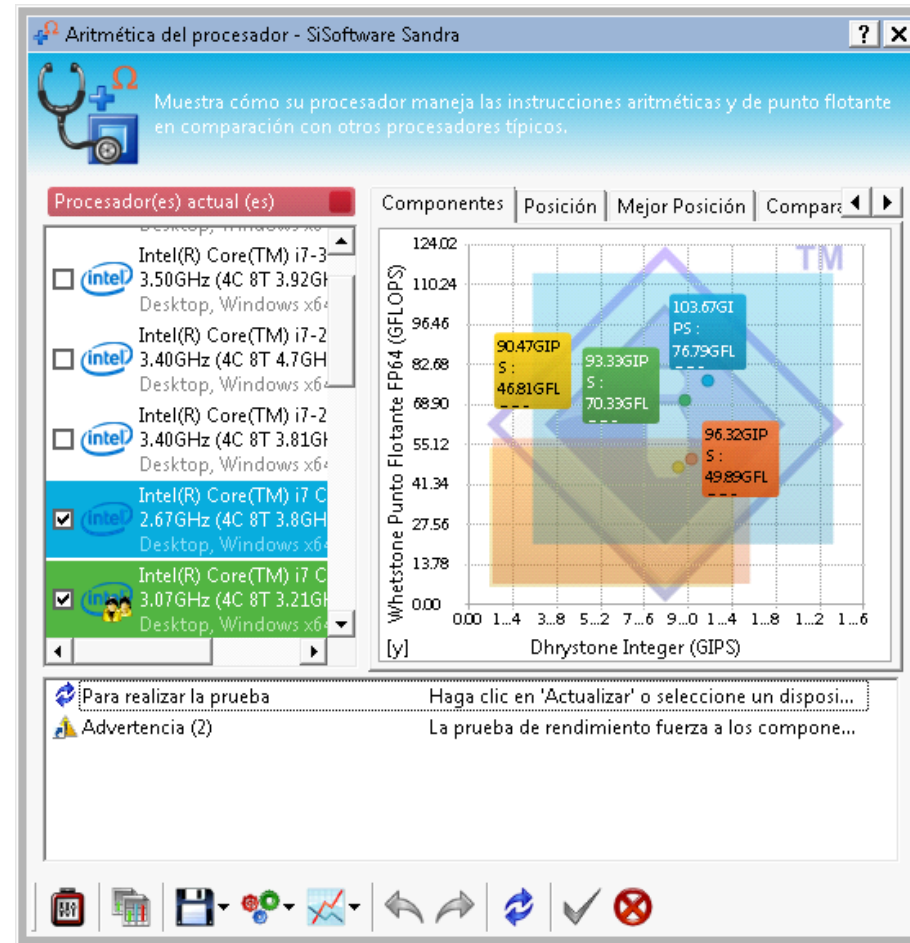
Passmark (Windows, <http://www.passmark.com/>),

Sandra (Windows, <http://www.sisoftware.net/>).

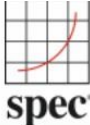
Paquetes de microbenchmarks: Sandra



Paquetes de microbenchmarks: Sandra



SPEC (<http://www.spec.org>)



Standard Performance Evaluation Corporation

- Home
- Benchmarks ▾
- Tools ▾
- Results ▾
- Contact
- Site Map
- Search
- Help

Benchmarks

- Cloud
- CPU
- Graphics/Workstations
- ACCEL/MPI/OMP
- Java Client/Server
- Mail Servers
- Storage
- Power
- Virtualization
- Web Servers

Results Search

Submitting Results

- Cloud/CPU/Java/Power
- SFS/Virtualization
- ACCEL/MPI/OMP
- SPECapc/SPECviewperf/SPECwpcc

Tools

- SERT
- PTDaemon
- Chauffeur WDK

Order Benchmarks

- Order Form

The Standard Performance Evaluation Corporation (SPEC) is a non-profit corporation formed to establish, maintain and endorse standardized benchmarks and tools to evaluate performance and energy efficiency for the newest generation of computing systems. SPEC develops benchmark suites and also reviews and publishes submitted results from our [member organizations](#) and other benchmark licensees.

What's New:

10/17/2017: The research and industrial/experience paper submission deadline for the [ICPE 2018](#) conference to be held April 9-13, 2018 in Berlin, has been extended to October 25, 2017. Please see the [important dates](#) for abstract and paper submission and notification schedules.

10/12/2017: The [SPECjbb2015](#) benchmark has been updated to version 1.01. This update adds support for Java SE 9. Licensees of version 1.0 are entitled to a complimentary V1.01 upgrade. SPEC will be retiring SPECjbb2015 V1.0 in January 2018, after which time all result submissions must be made with V1.01.

09/27/2017: SPEC's Application Performance Characterization (SPECapc) project group has released all-new performance evaluation software for systems running Autodesk Maya 2017 3D animation software. [SPECapc for Maya 2017 software](#) includes nearly 80 individual tests that reflect the processes used to model, animate and render scenes within the application. The new benchmark was developed by SPECapc in cooperation with Autodesk, which supplied seven new graphics models and scenes.

08/14/2017: With the release of [SPEC CPU2017](#), SPEC will be retiring SPEC CPU2006 in January 2018. Full details on the retirement schedule can be found [here](#).

El paquete de microbenchmarks SPEC CPU 2017

Compuesto por cuatro conjuntos de benchmarks (suites) con los que obtener 4 índices distintos (<http://www.spec.org/cpu2017/>):

SPECspeed[®]2017 Integer (rendimiento en aritmética entera)

SPECspeed[®]2017 Floating Point (rendimiento en coma flotante)

SPECrate[®]2017 Integer

SPECrate[®]2017 Floating Point

Speed: cuánto tarda en ejecutarse un programa (tiempo de respuesta)

Rate: cuántos programas puedo ejecutar por unidad de tiempo (productividad)

¿Qué componentes se evalúan?

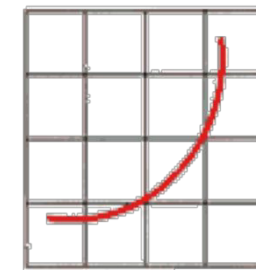
Procesador

Sistema de memoria

Compilador (C, Fortran y C++)

Reglas estrictas para validar los resultados:

<https://www.spec.org/cpu2017/Docs/runrules.html>



spec

SPEC CPU 2017 - suites

SPEC CPU2017 incluye 4 suites con un total de 43 benchmarks enfocadas en diferentes tipos de rendimiento intensivo de cómputo:

Criterios generales:

- Han de ser aplicaciones reales.

- Portabilidad a muchas arquitecturas: Intel y AMD x86 & x86-64, Sun SPARC, IBM POWER e IA-64.

Short Tag	Suite	Contents	Metrics
intspeed	SPECspeed 2017 Integer	10 integer benchmarks	SPECspeed2017_int_base SPECspeed2017_int_peak
fpspeed	SPECspeed 2017 Floating Point	10 floating point benchmarks	SPECspeed2017_fp_base SPECspeed2017_fp_peak
intrate	SPECrate 2017 Integer	10 integer benchmarks	SPECrate2017_int_base SPECrate2017_int_peak
fprate	SPECrate 2017 Floating Point	13 floating point benchmarks	SPECrate2017_fp_base SPECrate2017_fp_peak

SPEC CPU 2017 - suites

SPECrate 2017 Integer	SPECspeed 2017 Integer	Language[1]	KLOC[2]	Application Area
500.perlbench_r	600.perlbench_s	C	362	Perl interpreter
502.gcc_r	602.gcc_s	C	1,304	GNU C compiler
505.mcf_r	605.mcf_s	C	3	Route planning
520.omnetpp_r	620.omnetpp_s	C++	134	Discrete Event simulation - computer network
523.xalancbmk_r	623.xalancbmk_s	C++	520	XML to HTML conversion via XSLT
525.x264_r	625.x264_s	C	96	Video compression
531.deepsjeng_r	631.deepsjeng_s	C++	10	Artificial Intelligence: alpha-beta tree search (Chess)
541.leela_r	641.leela_s	C++	21	Artificial Intelligence: Monte Carlo tree search (Go)
548.exchange2_r	648.exchange2_s	Fortran	1	Artificial Intelligence: recursive solution generator (Sudoku)
557.xz_r	657.xz_s	C	33	General data compression

SPEC CPU 2017 - suites

SPECrate 2017 Floating Point	SPECspeed 2017 Floating Point	Language[1]	KLOC[2]	Application Area
503.bwaves_r	603.bwaves_s	Fortran	1	Explosion modeling
507.cactuBSSN_r	607.cactuBSSN_s	C++, C, Fortran	257	Physics: relativity
508.namd_r		C++	8	Molecular dynamics
510.parest_r		C++	427	Biomedical imaging: optical tomography with finite elements
511.povray_r		C++, C	170	Ray tracing
519.lbm_r	619.lbm_s	C	1	Fluid dynamics
521.wrf_r	621.wrf_s	Fortran, C	991	Weather forecasting
526.blender_r		C++, C	1,577	3D rendering and animation
527.cam4_r	627.cam4_s	Fortran, C	407	Atmosphere modeling
	628.pop2_s	Fortran, C	338	Wide-scale ocean modeling (climate level)
538.imagick_r	638.imagick_s	C	259	Image manipulation
544.nab_r	644.nab_s	C	24	Molecular dynamics
549.fotonik3d_r	649.fotonik3d_s	Fortran	14	Computational Electromagnetics
554.roms_r	654.roms_s	Fortran	210	Regional ocean modeling

©José Manuel Soto Hidalgo

Evaluación de Sistemas Informáticos. Referenciación

El paquete de microbenchmarks SPEC CPU 2017

SPEC CPU2017 se distribuye como una imagen ISO que contiene:

- Código fuente para los benchmarks de cada suite.

- Data sets que necesitan algunos benchmarks para su ejecución.

- Herramientas varias para compilación, ejecución, validación y generación de informes.

- Documentación, incluyendo reglas de ejecución y de generación de informes.

El tiempo de ejecución depende del índice a obtener, la máquina en la que se ejecuta y cuántas copias o subprocesos se eligen.

Metric	Config Tested	Individual benchmarks	Full Run (Reportable)
-----	-----	-----	-----
SPECrate2017_int_base	1 copy	6 to 10 minutes	2.5 hours
SPECrate2017_fp_base	1 copy	5 to 36 minutes	4.8 hours
SPECspeed2017_int_base	4 threads	6 to 15 minutes	3.1 hours
SPECspeed2017_fp_base	16 threads	6 to 75 minutes	4.7 hours

Índices de prestaciones en SPEC CPU2017

Índices de prestaciones (índices SPEC)

Aritmética entera: CPU2017IntegerSpeed_**peak**, CPU2017IntegerSpeed_**base**,
CPU2017IntegerRate_**peak**, CPU2017IntegerRate_**base**.

Aritmética en coma flotante: CPU2017FP_Speed_**peak**, CPU2017FP_Speed_**base**,
CPU2017FP_Rate_**peak**, CPU2017FP_Rate_**base**.

Significado de “base” y “peak”:

Base: Compilación en modo conservador, es decir, con reglas estrictas para que todos usen las mismas opciones de compilación.

Peak: Rendimiento pico, permitiendo que cada uno escoja las opciones de compilación óptimas para cada programa.

Cálculo

Cada programa del benchmark se ejecuta 3 veces y se escoge el resultado intermedio (se descartan los 2 extremos). El índice final es la media geométrica de esos tiempos de ejecución normalizados respecto a una máquina de referencia (Sun Fire V490 con procesador UltraSPARC IV+).

Resultados SPEC CPU2017



All SPEC CPU2017 Results Published by SPEC

These results have been submitted to SPEC; see [the disclaimer](#) before studying any results.

[Search published CPU2017 results](#)

Last update: 2017-10-19 11:49

[Search](#) | [CPU2017 Floating Point Rates](#) | [CPU2017 Floating Point Speed](#) | [CPU2017 Integer Rates](#) | [CPU2017 Integer Speed](#) |

CPU2017 Floating Point Rates (16):

[\[Search in CPU2017 Floating Point Rates results\]](#)

Test Sponsor	System Name	Base Copies	Processor			Results		Energy	
			Enabled Cores	Enabled Chips	Threads/ Core	Base	Peak	Base	Peak
Dell Inc.	PowerEdge R930 (Intel Xeon E7-8890 v4, 2.20 GHz) HTML CSV Text PDF PS Config	192	96	4	2	316	Not Run		--
Fujitsu	PRIMERGY RX2560 M2, Intel Xeon E5-2699A v4, 2.40GHz HTML CSV Text PDF PS Config	88	44	2	2	149	Not Run		--
Fujitsu	PRIMEQUEST 3800B, Intel Xeon Platinum 8180, 2.50GHz HTML CSV Text PDF PS Config	447	224	8	2	981	Not Run		--
Fujitsu	PRIMERGY RX2540 M4, Intel Xeon Platinum 8180, 2.50GHz HTML CSV Text PDF PS Config	112	56	2	2	247	Not Run		--
H3C	H3C R4900 G2 (Intel Xeon E5-2620 v4, 2.10 GHz) HTML CSV Text PDF PS Config	32	16	2	2	76.0	77.7		--
HPE	Integrity Superdome X (384 core, 2.20 GHz, Intel Xeon E7-8890 v4) HTML CSV Text PDF PS Config	768	384	16	2	1160	1170		--
HPE	ProLiant DL360 Gen9 (2.40 GHz, Intel Xeon E5-2699A v4) HTML CSV Text PDF PS Config	88	44	2	2	149	149		--
HPE	ProLiant DL380 Gen9 (2.40 GHz, Intel Xeon E5-2699A v4) HTML CSV Text PDF PS Config	88	44	2	2	147	148		--
	ProLiant DL380 Gen10 (2.10 GHz, Intel Xeon Platinum 8176)								

Resultados SPEC CPU2017



SPEC® CPU2017 Floating Point Rate Result

Copyright 2017 Standard Performance Evaluation Corporation

Dell Inc.

PowerEdge R930 (Intel Xeon E7-8890 v4, 2.20 GHz)

SPECrate2017_fp_base = 316

SPECrate2017_fp_peak = Not Run

CPU2017 License: 55

Test Sponsor: Dell Inc.

Tested by: Dell Inc.

Test Date: Dec-2016

Hardware Availability: Oct-2016

Software Availability: Nov-2016

Benchmark result graphs are available in the PDF report.

Hardware

CPU Name:

Intel Xeon E7-8890 v4

Max MHz:

3400

Nominal:

2200

Enabled:

96 cores, 4 chips, 2 threads/core

Orderable:

2,4 chips

Cache L1:

32 KB I + 32 KB D on chip per core

L2:

256 KB I+D on chip per core

L3:

60 MB I+D on chip per chip

Other:

None

Memory:

512 GB (32 x 16 GB 2Rx8 PC4-2133P-R, running at 1600 MHz)

Storage:

1 x 480 GB SATA SSD

Other:

None

Software

OS:

SUSE Linux enterprise Server 12 SP1 3.12.49-11-default

Compiler:

C/C++: Version 17.0.0.098 of Intel C/C++ Compiler for Linux;
Fortran: Version 17.0.0.098 of Intel Fortran Compiler for Linux

Parallel:

No

Firmware:

2.2.0

File System:

xfs

System State:

Run level 3 (multi-user)

Base Pointers:

32/64-bit

Peak Pointers:

Not Applicable

Other:

Microquill SmartHeap V10.2

Results Table

Benchmark	Base							Peak						
	Copies	Seconds	Ratio	Seconds	Ratio	Seconds	Ratio	Copies	Seconds	Ratio	Seconds	Ratio	Seconds	Ratio
503.bwaves_r	192	2741	703	2741	702	2742	702							
507.cactuBSSN_r	192	1136	214	1138	214	1138	214							
508.namd_r	192	572	319	570	320	570	320							
510.parest_r	192	2902	173	2899	173	2928	172							
511.povray_r	192	860	521	858	522	861	520							

CPU2017fp_base = $\sqrt[01]{702 \times 214 \times \dots}$

$$CPU2017fp_base = \sqrt[0]{702 \times 214 \times 320 \times \dots}$$

Resultados SPEC CPU2017



spec

SPEC® CPU2017 Floating Point Rate Result

Copyright 2017 Standard Performance Evaluation Corporation

H3C

H3C R4900 G2 (Intel Xeon E5-2620 v4, 2.10 GHz)

SPECrate2017_fp_base = 76.0

SPECrate2017_fp_peak = 77.7

CPU2017 License: 9066

Test Sponsor: H3C

Tested by: H3C

Test Date: Feb-2017

Hardware Availability: Oct-2016

Software Availability: Oct-2016

Benchmark result graphs are available in the PDF report.

Hardware

CPU Name: Intel Xeon E5-2620 v4

Max MHz: 3000

Nominal: 2100

Enabled: 16 cores, 2 chips, 2 threads/core

Orderable: 1,2 chip

Cache L1: 32 KB I + 32 KB D on chip per core

L2: 256 KB I+D on chip per core

L3: 20 MB I+D on chip per chip

Other: None

Memory: 256 GB (16 x 16 GB 2Rx4 PC4-2400T-R, running at 2133 MHz)

Storage: 500GB SATA 7200 RPM

Other: None

Software

OS: SUSE Linux Enterprise Server 12 SP1
SUSE Linux Enterprise Server 12 SP1
3.12.49-11-default

Compiler: C/C++: Version 17.0.0.098 of Intel C++ Compiler Professional Build 20160721;
Fortran: Version 17.0.0.098 of Intel Fortran Compiler Professional Build 20160721;

Parallel: No

Firmware: BIOS American Megatrends Inc. 1.00.15 10/17/2016

File System: xfs

System State: Run level 3 (multi-user)

Base Pointers: 32/64-bit

Peak Pointers: Not Applicable

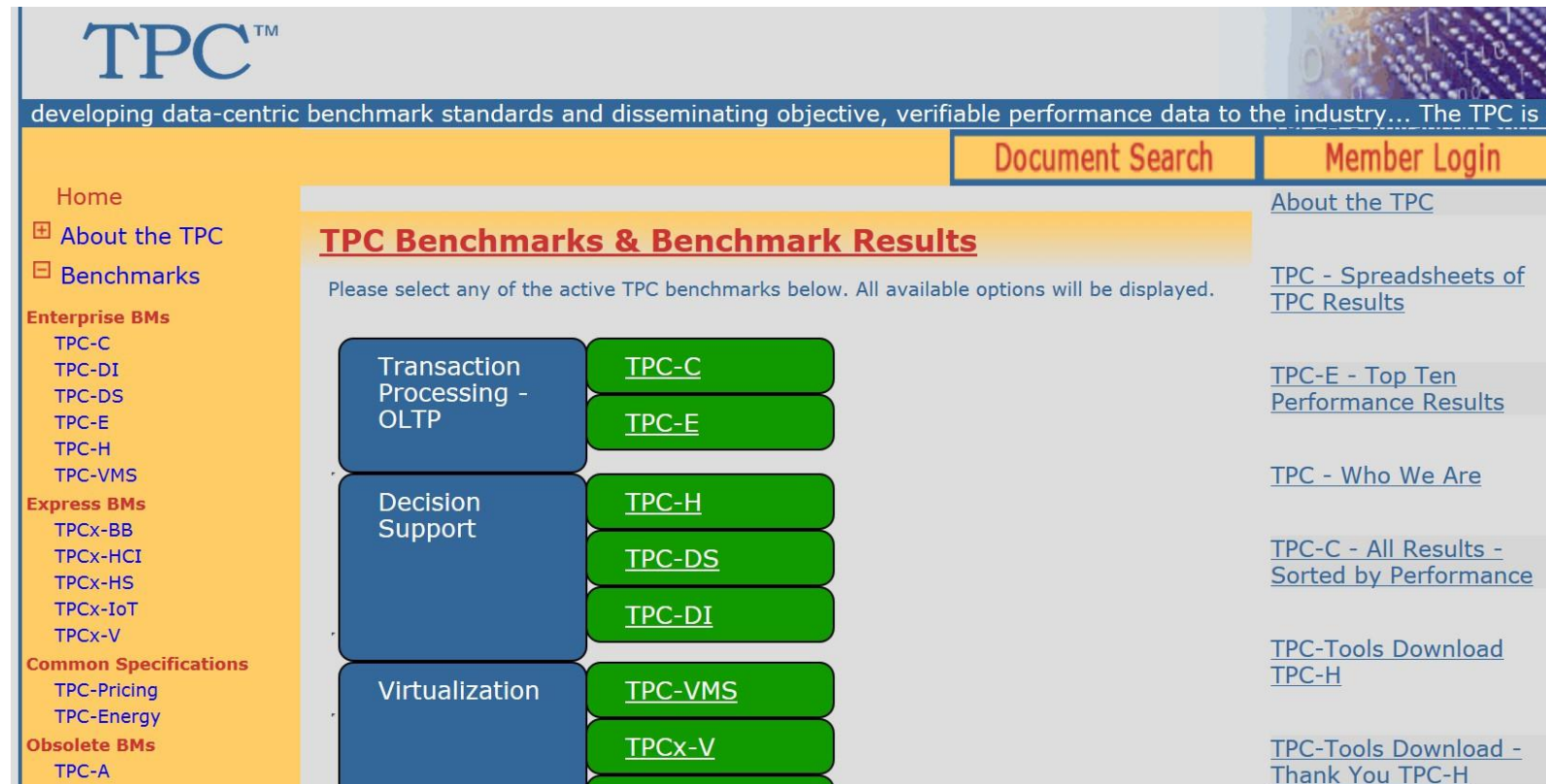
Other: None

Results Table

Benchmark	Base							Peak						
	Copies	Seconds	Ratio	Seconds	Ratio	Seconds	Ratio	Copies	Seconds	Ratio	Seconds	Ratio	Seconds	Ratio
503.bwaves_r	32	1301	247	1300	247	1296	248	32	1301	247	1300	247	1296	248
507.cactuBSSN_r	32	737	55.0	735	55.1	733	55.3	32	737	55.0	735	55.1	733	55.3
508.namd_r	32	588	51.7	589	51.6	590	51.5	32	582	52.2	582	52.2	583	52.2
510.parest_r	32	1535	54.6	1538	54.4	1535	54.6	32	1536	54.5	1526	54.9	1525	54.9
511.povray_r	32	892	83.8	892	83.7	890	83.9	32	753	99.2	742	101	745	100

Benchmarks de sistema completo: TPC

TPC (*Transactions Processing Performance Council*, <http://www.tpc.org>): Organización sin ánimo de lucro especializada en benchmarks relacionados con comercio electrónico y con bases de datos.



The screenshot shows the TPC website interface. At the top, the TPC logo is displayed with the tagline "developing data-centric benchmark standards and disseminating objective, verifiable performance data to the industry... The TPC is a". Below the logo, there are navigation links: "Home", "About the TPC", and "Benchmarks". The "Benchmarks" section is expanded, showing a list of benchmarks categorized into "Enterprise BMs", "Express BMs", "Common Specifications", and "Obsolete BMs". The "Enterprise BMs" list includes TPC-C, TPC-DI, TPC-DS, TPC-E, TPC-H, and TPC-VMS. The "Express BMs" list includes TPCx-BB, TPCx-HCI, TPCx-HS, TPCx-IoT, and TPCx-V. The "Common Specifications" list includes TPC-Pricing and TPC-Energy. The "Obsolete BMs" list includes TPC-A. The "Benchmarks & Benchmark Results" section is highlighted, showing a table of benchmarks and their results. The table has two columns: "Benchmark" and "Results". The benchmarks listed are Transaction Processing - OLTP, Decision Support, and Virtualization. The results for Transaction Processing - OLTP are TPC-C and TPC-E. The results for Decision Support are TPC-H, TPC-DS, and TPC-DI. The results for Virtualization are TPC-VMS and TPCx-V. On the right side of the page, there are links for "Document Search", "Member Login", "About the TPC", "TPC - Spreadsheets of TPC Results", "TPC-E - Top Ten Performance Results", "TPC - Who We Are", "TPC-C - All Results - Sorted by Performance", "TPC-Tools Download TPC-H", and "TPC-Tools Download - Thank You TPC-H".

TPC™

developing data-centric benchmark standards and disseminating objective, verifiable performance data to the industry... The TPC is a

Document Search Member Login

Home

About the TPC

Benchmarks

Enterprise BMs

- TPC-C
- TPC-DI
- TPC-DS
- TPC-E
- TPC-H
- TPC-VMS

Express BMs

- TPCx-BB
- TPCx-HCI
- TPCx-HS
- TPCx-IoT
- TPCx-V

Common Specifications

- TPC-Pricing
- TPC-Energy

Obsolete BMs

- TPC-A

TPC Benchmarks & Benchmark Results

Please select any of the active TPC benchmarks below. All available options will be displayed.

Transaction Processing - OLTP	TPC-C
	TPC-E
Decision Support	TPC-H
	TPC-DS
	TPC-DI
Virtualization	TPC-VMS
	TPCx-V

About the TPC

[TPC - Spreadsheets of TPC Results](#)

[TPC-E - Top Ten Performance Results](#)

[TPC - Who We Are](#)

[TPC-C - All Results - Sorted by Performance](#)

[TPC-Tools Download TPC-H](#)

[TPC-Tools Download - Thank You TPC-H](#)

Benchmarks de sistema completo: TPC

Principales benchmarks:

TPC-C: Tipo OLTP (*on-line transaction processing*). Simula una gran compañía con varios almacenes, cada uno con 100.000 productos y tiene 3000 clientes. Peticiones que involucran acceso a las bases de datos tanto locales como distribuidas.

TPC-E: Tipo OLTP. Simula una correduría de bolsa en donde hay una única base de datos central. El benchmark es escalable de modo que se pueden simular transacciones de compañías de diversos tamaños.

TPC-H, TPC-DS: Tipo DS (*decision support*). Se deben ejecutar consultas altamente complejas a una gran base de datos y analizar enormes volúmenes de datos.

Métricas: peticiones/transacciones procesadas por unidad de tiempo (*tps/tpm/tph*) superando unos ciertos requisitos de tiempos de respuesta. También: coste por petición procesada (incluido mantenimiento) y consumo de potencia por petición procesada.

TPC-C: Los mejores resultados



TPC-C - Top Ten Performance Results

Version 5 Results As of 18-Mar-2014 12:40 PM [GMT]

Note 1: The TPC believes it is not valid to compare prices or price/performance of results in different currencies.

☐ All Results
 ☐ Clustered Results
 ☐ Non-Clustered Results
 Currency All

Rank	Company	System	Performance (tpmC)	Price/tpmC	Watts/KtpmC	System Availability	Database	Operating System	TP Monitor	Date Submitted	Cluster
1	ORACLE	SPARC T5-8 Server	8,552,523	.55 USD	NR	09/25/13	Oracle 11g Release 2 Enterprise Edition with Oracle Partitioning	Oracle Solaris 11.1	Oracle Tuxedo CFSR	03/26/13	N
2	ORACLE	Sun Server X2-8	5,055,888	.89 USD	NR	07/10/12	Oracle Database 11g R2 Enterprise Edition w/Partitioning	Oracle Linux w/Unbreakable Enterprise Kernel R2	Tuxedo CFS-R	03/27/12	N
3	IBM	IBM System x3850 X5	3,014,684	.59 USD	NR	09/22/11	IBM DB2 ESE 9.7	SUSE Linux Enterprise Server 11 SP1 for X86_64	Microsoft COM+	07/11/11	N
4	CISCO	Cisco UCS C240 M3 Rack Server	1,609,186	.47 USD	NR	09/27/12	Oracle Database 11g Standard Edition One	Oracle Linux w/Unbreakable Enterprise Kernel R2	Microsoft COM+	09/27/12	N
5	IBM	IBM Flex System x240	1,503,544	.53 USD	NR	08/16/12	IBM DB2 ESE 9.7	Red Hat Enterprise Linux 6.2	Microsoft COM+	04/11/12	N
6	IBM	IBM System x3650 M4	1,320,082	.51 USD	NR	02/25/13	IBM DB2 ESE 9.7	Red Hat Enterprise Linux 6.4 with KVM	Microsoft COM+	02/22/13	N
	HP	HP ProLiant Blade					Microsoft SQL Server	Microsoft Windows	Microsoft		

Evaluación de Sistemas Informáticos. Referenciación

Benchmarks de sistema completo: SPEC

File Server: SFS2014: Tiempos de respuesta y productividades de servidores de ficheros.

High Performance Computing, OpenMP, MPI, OpenCL

SPEC MPI2007: Message Passing Interface (MPI).

SPEC OMP2012: Open MultiProcessing (OpenMP).

SPEC ACCEL: OpenCL y OpenACC

JAVA Cliente/Servidor

SPECjEnterprise2010: Java Enterprise Edition (JEE).

SPECjms2007: Java Message Service (JMS).

SPECjvm2008: Java Runtime Environment (JRE).

Virtualization: SPECvirt_sc2010 (Virtualización en Centros de Procesamiento de Datos).

Cloud: SPEC Cloud_IaaS 2016 (Servicios en la nube)

Consumo de potencia: SPECpower_ssj2008 (Rendimiento de un servidor ejecutando aplicaciones JAVA frente al consumo de potencia).

Benchmarks de sistema completo: SYSMark 2012

Para comparar PC con S.O. Windows.

Considera la carga en 6 escenarios:

Office Productivity: Word, PowerPoint, Outlook, Acrobat ...

Media Creation: Adobe Photoshop, Adobe Premiere...

Web Development: Dreamweaver, IE, Firefox...

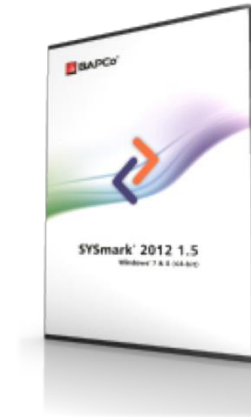
Data/Financial Analysis: Excel.

3D Modeling: Autodesk 3ds Max, AutoCAD, Google SketchUp...

System Management: Winzip, Firefox installer.

Con cada programa se ejecuta un conjunto de tareas de acuerdo con un modelo de comportamiento de un usuario “habilitado”.

El tiempo medio de ejecución de los benchmarks de cada categoría se normaliza (ratio) respecto de una máquina de referencia. Finalmente, el índice SYSMark2012 se calcula mediante la media geométrica de los ratios obtenidos.



Análisis de resultados

¿Cómo expresar el rendimiento?

El rendimiento es una variable multidimensional

Habría de expresarse mediante múltiples índices

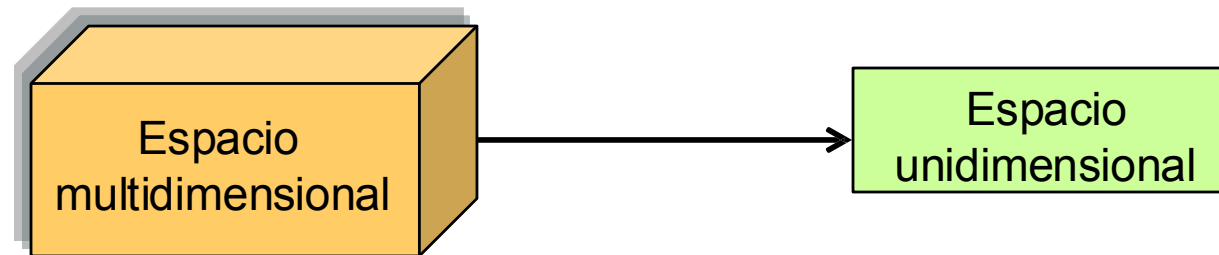
Sin embargo, las comparaciones son más sencillas si se usa un único índice de rendimiento (a minimizar o maximizar)

¿Cómo concentrar todas las variables en una sola?

Utilizar la *mejor* variable que represente el rendimiento

Asegurar que su obtención es válida

Método habitual de síntesis: uso de **medias**



La media aritmética

Dado un conjunto de n medidas, $t_1, \dots, t_n$, definimos su media aritmética:

$$\bar{t} = \frac{1}{n} \sum_{k=1}^n t_k$$

Si no todas las medidas tienen la misma importancia, se puede asociar a cada medida t_k un peso w_k , obteniéndose la **media aritmética ponderada**:

$$\bar{t}_W = \sum_{k=1}^n w_k \times t_k$$

$$\text{con } \sum_{k=1}^n w_k = 1$$

Si t_k es el tiempo de ejecución del programa de benchmark k -ésimo, w_k podría escogerse, por ejemplo, inversamente proporcional a t_{REF_k} , el tiempo de ejecución en la máquina de referencia:

$$w_k \equiv \frac{C}{t_{REF_k}}$$
$$C = \frac{1}{\sum_{k=1}^n 1/t_{REF_k}}$$

La media geométrica

Dado un conjunto de n medidas, $r_1, \dots, r_n$, definimos su media geométrica:

$$\bar{r}_g = \sqrt[n]{\prod_{k=1}^n r_k} = \left(\prod_{k=1}^n r_k \right)^{1/n}$$

Propiedad: cuando los valores son medidas de ganancias en velocidad con respecto a un sistema de referencia, este índice mantiene el mismo orden en las comparaciones independientemente del sistema de referencia usado. Usado en los benchmarks de SPEC y SYSMARK.

$$SPEC(M) = \sqrt[n]{\frac{t_1^{REF}}{t_1^M} \times \frac{t_2^{REF}}{t_2^M} \times \dots \times \frac{t_n^{REF}}{t_n^M}} = \frac{\sqrt[n]{t_1^{REF} \times t_2^{REF} \times \dots \times t_n^{REF}}}{\sqrt[n]{t_1^M \times t_2^M \times \dots \times t_n^M}}$$

$$SPEC(M1) > SPEC(M2) \Leftrightarrow \sqrt[n]{t_1^{M1} \times t_2^{M1} \times \dots \times t_n^{M1}} < \sqrt[n]{t_1^{M2} \times t_2^{M2} \times \dots \times t_n^{M2}}$$

Ejemplo de comparación con tiempos

Programa	Ref. (s)	A (s)	B (s)	C (s)	D (s)
1	1400	141	170	136	134
2	1400	154	166	215	25
3	1100	96,8	94,2	146	201
4	1800	271	283	428	523
5	1000	83,8	90,1	77,4	81,2
6	1200	179	189	199	245
7	1300	120	131	87,7	75,5
8	300	151	158	138	192
9	1100	93,5	122	88	118
10	1900	133	173	118	142
11	1500	173	170	179	240
12	3000	243	100	100	150
Suma	17000,0	1839,1	1846,3	1912,1	2126,7

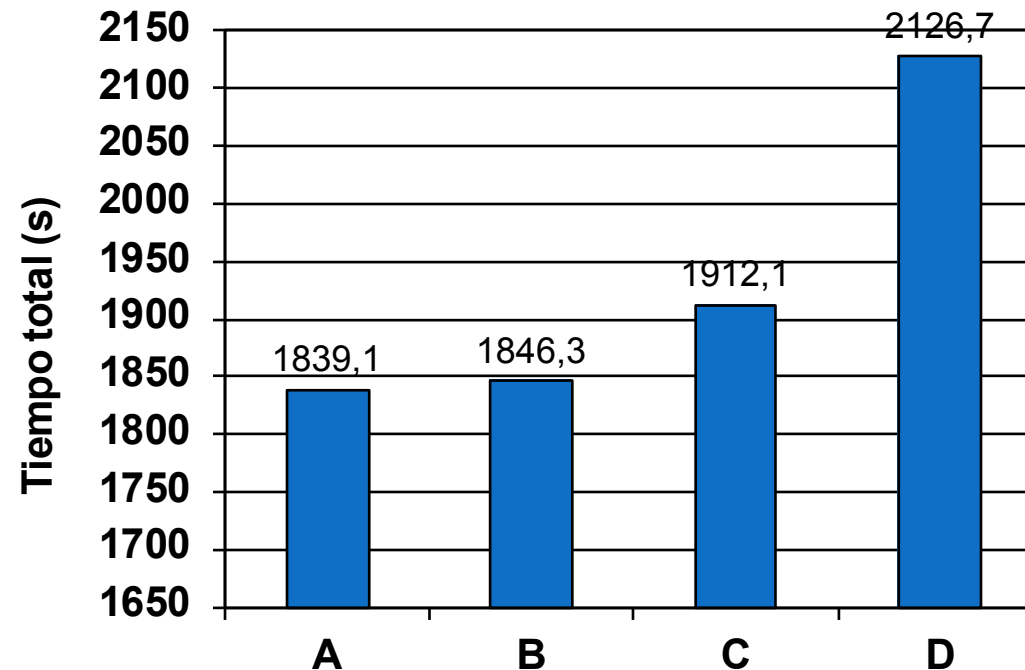
La máquina más rápida es “A” ya que es la que tarda menos en ejecutar todos los programas del benchmark (1839,1 segundos).

Comparación con el tiempo total

Ordenación con el tiempo total:

De más rápida a más lenta: A, B, C, D

Esto no significa que A sea siempre la más rápida (depende del programa), aunque, en conjunto, sí que lo es.



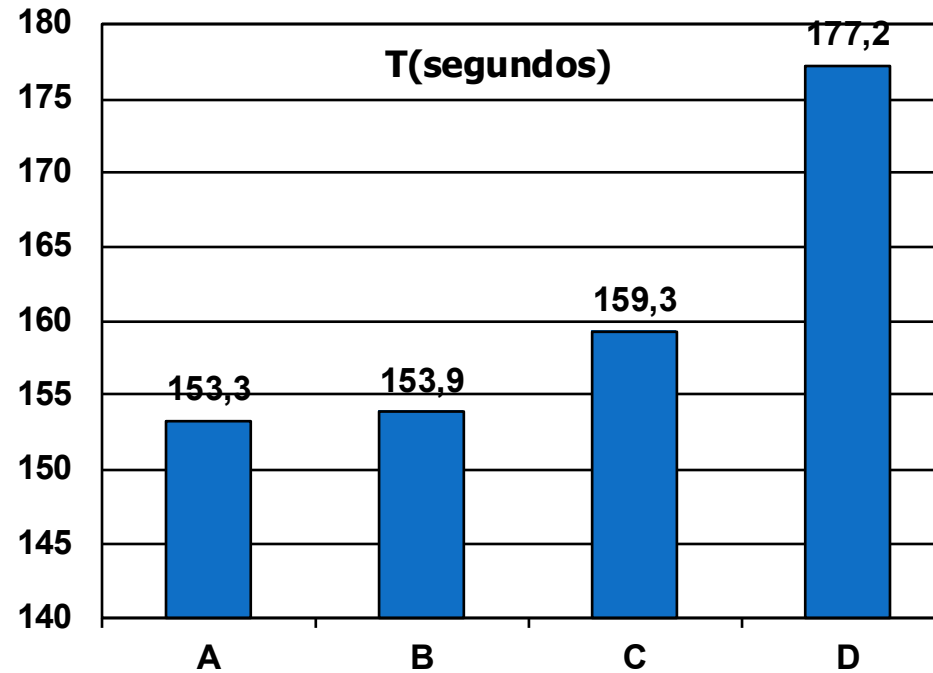
Comparación con la media aritmética

$$\bar{x}_A = \frac{1}{12} \sum_{i=1}^{12} t_{A,i} = \frac{1839,1}{12} = 153,3s$$

$$\bar{x}_B = \frac{1}{12} \sum_{i=1}^{12} t_{B,i} = \frac{1846,3}{12} = 153,9s$$

$$\bar{x}_C = \frac{1}{12} \sum_{i=1}^{12} t_{C,i} = \frac{1912,1}{12} = 159,3s$$

$$\bar{x}_D = \frac{1}{12} \sum_{i=1}^{12} t_{D,i} = \frac{2126,7}{12} = 177,2s$$



La máquina más rápida es la de mejor tiempo de ejecución medio (media aritmética de los tiempos de ejecución).

Usando la media aritmética ponderada

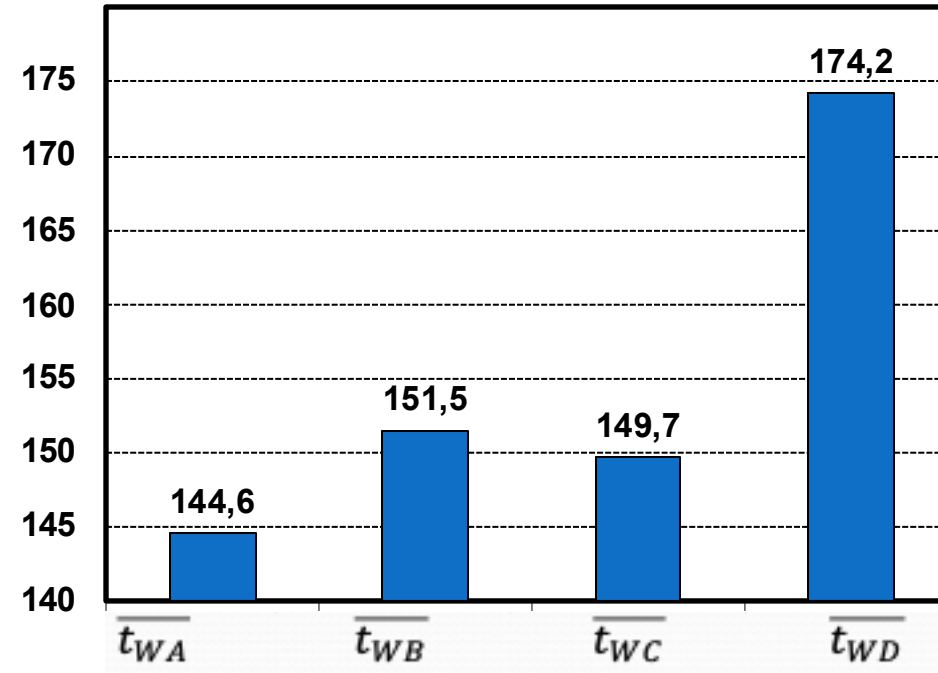
Prog	Ref. (s)	w _k
1	1400	0,06
2	1400	0,06
3	1100	0,08
4	1800	0,05
5	1000	0,09
6	1200	0,07
7	1300	0,07
8	300	0,30
9	1100	0,08
10	1900	0,05
11	1500	0,06
12	3000	0,03
Suma	17000	1

$$C = \frac{1}{\sum_{k=1}^n 1/t_{REF_k}} = 88,77s$$

$$w_k \equiv \frac{C}{t_{REF_k}}$$

$$\overline{t_{WA}} = \frac{1}{12} \sum_{k=1}^{12} w_k \times t_{A,k} = 144,6s$$

Igualmente, se calculan:
 $\overline{t_{WB}}$, $\overline{t_{WC}}$ y $\overline{t_{WD}}$



Esta ponderación depende, en este ejemplo, de la máquina de referencia.

Normalización de rendimientos: ratios

Cada columna es el cociente entre el tiempo de ejecución de la máquina de referencia y el de la máquina a comparar (SPEC y Sysmark):

Programa	Ref (s)	A (ratio)	B (ratio)	C (ratio)	D (ratio)
1	1400	9,9	8,2	10,3	10,4
2	1400	9,1	8,4	6,5	56,0
3	1100	11,4	11,7	7,5	5,5
4	1800	6,6	6,4	4,2	3,4
5	1000	11,9	11,1	12,9	12,3
6	1200	6,7	6,3	6,0	4,9
7	1300	10,8	9,9	14,8	17,2
8	300	2,0	1,9	2,2	1,6
9	1100	11,8	9,0	12,5	9,3
10	1900	14,3	11,0	16,1	13,4
11	1500	8,7	8,8	8,4	6,3
12	3000	12,3	30,0	30,0	20,0
M. Geom.		8,78	8,66	8,97	9,00

- El ratio (=ganancia en velocidad) es un índice a maximizar

¿Por qué se usa la media geométrica?

Se premian las mejoras sustanciales. No se castigan empeoramientos no tan sustanciales. Debemos ser MUY cuidadosos con las comparaciones y saber qué estamos haciendo realmente.

J8 =MEDIA.GEOM(J2:J5)										
	A	B	C	D	E	F	G	H	I	J
	Prog. Bench.	Tiempo Ejecución Máquina	A(s)	B(s)	C(s)	D(s)	R/A	R/B	R/C	R/D
1		Ref (s)								
2	1	200	100	99	1	1	2,00	2,02	200,0	200,0
3	2	200	100	101	133	1	2,00	1,98	1,50	200,0
4	3	200	100	100	133	1	2,00	2,00	1,50	200,0
5	4	200	100	100	133	397	2,00	2,00	1,50	0,50
6	Suma	800	400	400	400	400				
7										
8				Media Geométrica			2,0000	2,0001	5,11	44,81

¿Por qué se usa la media geométrica?

Veamos el siguiente caso de estudio

Tres sistemas: A, B y C

Dos programas de prueba: P1 y P2

Tiempo de ejecución en segundos de cada programa:

	A	B	C
P1	1	10	20
P2	1000	100	20

Para este benchmark, la máquina más rápida es “C” que tarda un total de 40 segundos en ejecutar el benchmark completo.

C es $1001/40 = 25$ veces más rápida que A en ejecutar el benchmark

C es $110/40 = 2,75$ veces más rápida que B en ejecutar el benchmark

B es $1001/110 = 9,1$ veces más rápida que A en ejecutar el benchmark

¿Por qué se usa la media geométrica?

Vamos a calcular ratios normalizando según diferentes máquinas de referencia.

	A	B	C
P1	1	10	20
P2	1000	100	20

A mayor ratio,
mejor

	Referencia = A			Referencia = B			Referencia = C		
	A	B	C	A	B	C	A	B	C
Programa P1	1.0	0.1	0.05	10.0	1.0	0.5	20.0	2.0	1.0
Programa P2	1.0	10.0	50.0	0.1	1.0	5.0	0.02	0.2	1.0
Media aritmética	1.0	5.05	25.03	5.05	1.0	2.25	10.01	1.1	1.0
Media geométrica	1.0	1.0	1.58	1.0	1.0	1.58	0.63	0.63	1.0

La media geométrica siempre obtiene el mismo orden entre las máquinas, independientemente de la máquina de referencia: es una medida consistente/robusta ante el cambio de referencia. **¿Es una razón suficiente?**

Conclusiones de este análisis

Intentar reducir un conjunto de medidas de un benchmark a un solo “valor medio” final no es una tarea trivial.

La media aritmética de los tiempos de ejecución de un benchmark es una medida fácilmente interpretable e independiente de ninguna máquina de referencia. El menor valor nos indica la máquina que ha ejecutado el **conjunto** de programas del benchmark en un tiempo menor.

La media aritmética ponderada nos permite asignar más peso a algunos programas que a otros. Esa ponderación debería realizarse, idealmente, según las necesidades del usuario. Si se hace de forma dependiente de los tiempos de ejecución de una máquina de referencia, la elección de ésta puede influir significativamente en los resultados.

La media geométrica de las ganancias en velocidad con respecto a una máquina de referencia es un índice de interpretación complejo que no depende la máquina de referencia. Premia mejoras sustanciales con respecto a algún programa del benchmark y no castiga al mismo nivel los empeoramientos.

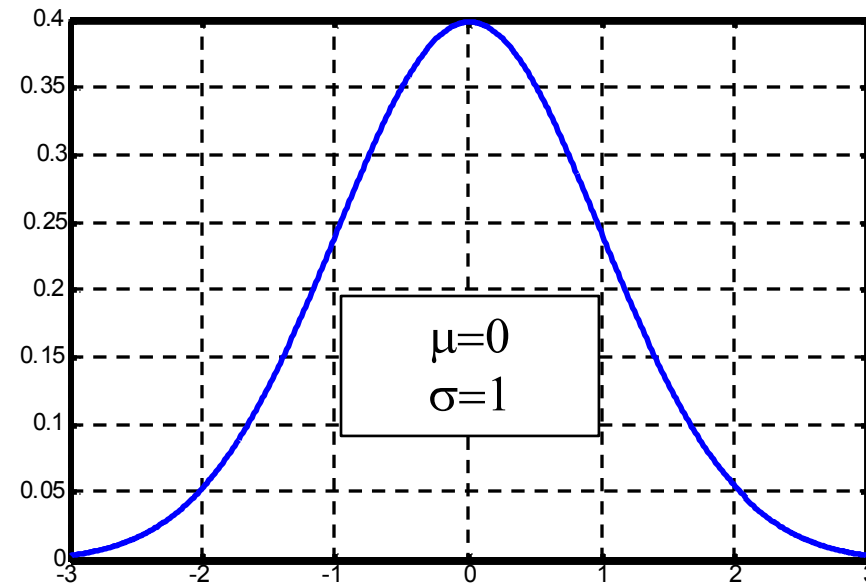
Independientemente de qué índice se escoja, un buen ingeniero debería en primer lugar determinar si las diferencias entre las diferentes medidas obtenidas son **estadísticamente significativas**. ¿Qué significa eso?

Repaso de estadística: Distribución Normal

Es una distribución caracterizada por su media μ y su dispersión σ^2 cuya función de probabilidad viene dada por:

$$Prob(x) = \frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{(x-\mu)^2}{2\sigma^2}}$$

La probabilidad de obtener un elemento en el rango $[\mu - 2\sigma, \mu + 2\sigma]$ es del 95%



Teorema del límite central: la suma de un conjunto de muestras aleatorias pertenecientes a cualquier distribución e independientes entre sí tiende a una distribución normal.

Repaso de Estadística: Distribución t de Student

Si disponemos de n muestras d_i pertenecientes a una distribución Normal de media $\bar{d}_{real}$, el número (=estadístico):

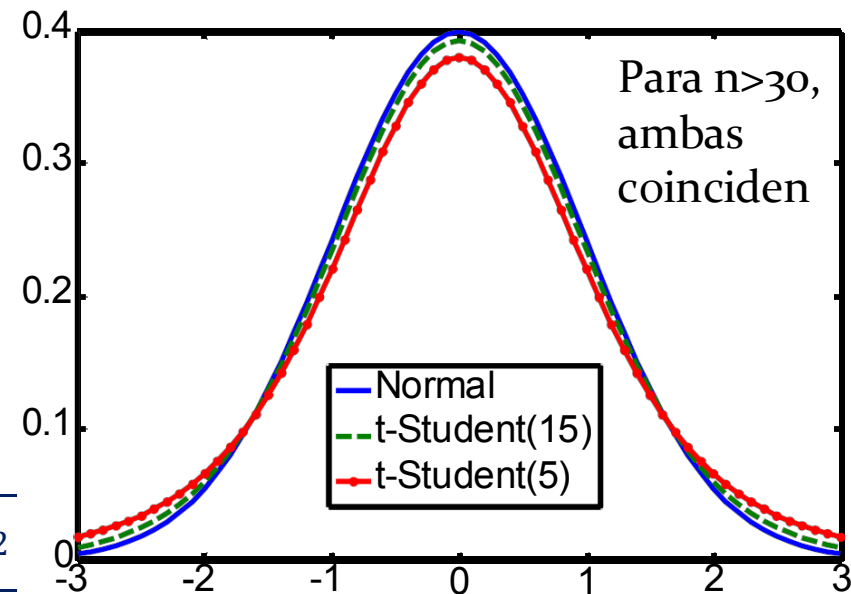
$$t_{exp} = \frac{\bar{d} - \bar{d}_{real}}{s/\sqrt{n}}$$

siendo $\bar{d}$ la media muestral:

$$\bar{d} = \frac{\sum_{i=1}^n d_i}{n}$$

y s la desviación típica muestral:

$$s = \sqrt{\frac{\sum_{i=1}^n (d_i - \bar{d})^2}{n-1}} = \sqrt{\frac{\sum_{i=1}^n d_i^2 - n \cdot \bar{d}^2}{n-1}}$$



se distribuye según la distribución t-Student con $n-1$ grados de libertad. **¿Para qué me puede servir esto?**

Temario

Ejemplo 1

- Tiempos de ejecución (en segundos) de un benchmark compuesto por 6 programas (P1...P6) en dos máquinas diferentes (A y B)

Programa	A (s)	B (s)	$d_i = tA_i - tB_i$
P1	142	100	42
P2	139	92	47
P3	152	128	24
P4	112	82	30
P5	156	148	8
P6	166	171	-5
Suma	867	721	$\bar{d}$

¿Significativas diferencias?

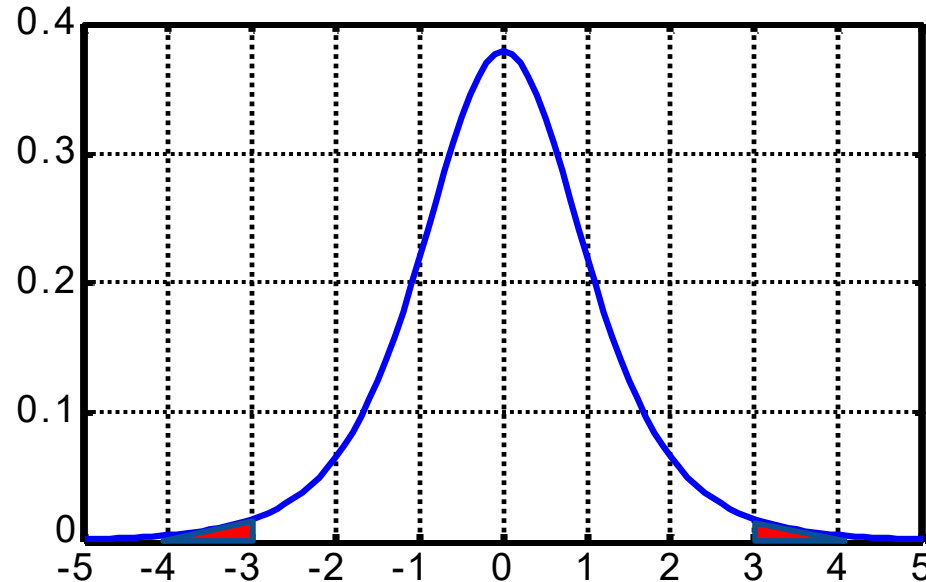
est $\bar{d} = 24,3 \text{ seg}$
 $s = 19,9 \text{ seg}$

- Si partimos de la hipótesis de que las máquinas son iguales, entonces las diferencias se deben a factores aleatorios independientes. En ese caso d_i serán muestras de una distribución normal de media cero. Por tanto:

$$t = \frac{\bar{d}}{s/\sqrt{n}} = 2,99$$

pertenece a una distribución t-student con 6-1=5 grados de libertad.
¿Qué probabilidad hay de que esto sea realmente así?

Distribución t-student con 5 grados de libertad:



$$t_{exp} = \frac{\bar{d}}{s/\sqrt{n}} = 2,99$$

$$P - value = P(|t| \geq 2,99; 5) \\ = 2 \times P(t < -2,99; 5)$$

$$= 2 \cdot \text{tcdf}(-2.99, 5) = 0.0304 \\ (\text{Matlab})$$

$$= \text{DISTR.T.2C}(2,99; 5) = 0.0304 \\ (\text{Excel})$$

La probabilidad de obtener un valor de $|t|$ igual o superior a 2,99 de una distribución t-student con 5 grados de libertad es de 3,04 %. (**P-value**= 0.0304).

¿Es eso mucho o poco? Debemos definir un umbral: **nivel de significatividad** α .

Conclusión: Si $P\text{-value} < \alpha$ diremos que, para un grado de significatividad α o para un **nivel de confianza** $(1-\alpha)*100=95\%$, las máquinas tienen rendimientos estadísticamente diferentes.

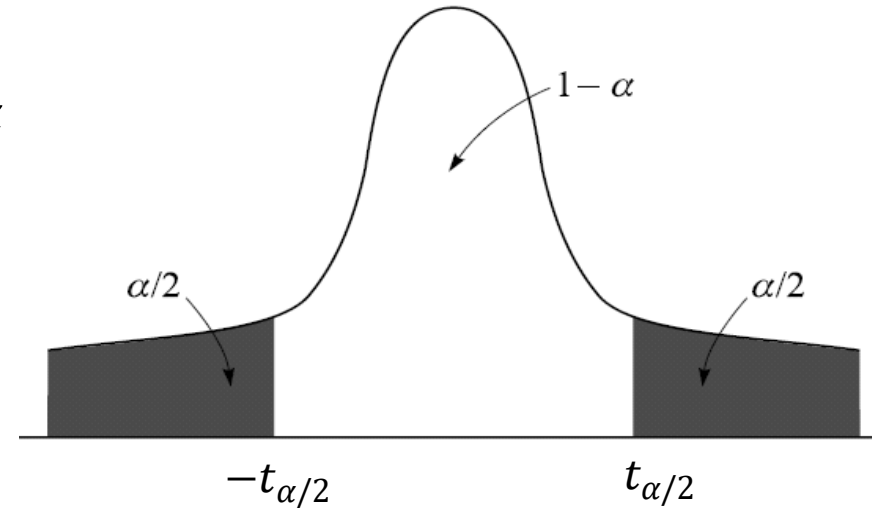
Intervalos de confianza

Para un **nivel de significatividad** α (típ. 0.05), buscamos el valor $t_{\alpha/2}$ que cumpla:

$$Prob(-t_{\alpha/2} \leq t \leq t_{\alpha/2}) = 1 - \alpha$$

Diremos que para un **nivel de confianza** $(1-\alpha) \times 100$ (típ. 95%), el valor de **t** debe situarse en el intervalo:

$$[-t_{\alpha/2}, t_{\alpha/2}]$$



A dicho intervalo se le denomina **intervalo de confianza** de la medida, para un nivel de significatividad α . Teniendo en cuenta que:

$$Prob(-t_{\alpha/2} \leq t \leq t_{\alpha/2}) = 1 - 2 \times Prob(t \leq -t_{\alpha/2}) = 1 - 2 \times Prob(t > t_{\alpha/2})$$

es fácil demostrar que $t_{\alpha/2}$ cumple que (ver figura):

$$Prob(t \leq -t_{\alpha/2}) = Prob(t > t_{\alpha/2}) = \alpha/2$$

Intervalos de confianza para t_{exp}

En el caso del *Ejemplo 1*, para un nivel de significatividad de $\alpha=0.05$, buscamos $t_{\alpha/2, n-1}$ tal que:

$$Prob\left(t \leq -t_{\frac{\alpha}{2}, n-1}\right) = \alpha/2 = 0.025$$

para una distribución t de Student con 5 grados de libertad. Eso se puede obtener, por ejemplo:

En Matlab, haciendo: $\text{abs}(\text{tinv}(\alpha/2, n-1)) = \text{abs}(\text{tinv}(0.025, 5)) = 2.57$

En Excel, haciendo: $\text{ABS}(\text{INV.T}(\alpha/2; n-1)) = \text{ABS}(\text{INV.T}(0.025; 5)) = 2.57$ o

directamente: $\text{INV.T.2C}(\alpha; n-1) = \text{INV.T.2C}(0.05; 5) = 2.57$

Dicho de otra manera, si las diferencias entre los tiempos de ejecución de ambas máquinas se debieran a factores aleatorios, existiría un 95% de probabilidad de que

$$t_{exp} = \frac{\bar{d}}{s/\sqrt{n}}$$

se encuentre en el rango $[-t_{\frac{\alpha}{2}, n-1}, t_{\frac{\alpha}{2}, n-1}] = [-t_{0.025, 5}, t_{0.025, 5}] = [-2.57, 2.57]$.

➡ Como $t_{exp}=2.99$ no está en ese rango, concluiremos que la hipótesis de que ambas máquinas pueden ser iguales **no** es cierta al 95% de confianza.

Intervalos de confianza

- Valores de la distribución t-student para $\alpha=0.05$ (nivel de confianza del 95%) con **n-1** grados de libertad

n-1	$t_{0.025, n-1}$	n-1	$t_{0.025, n-1}$
1	12.706	7	2.365
2	4.303	8	2.306
3	3.182	9	2.262
4	2.776	10	2.228
5	2.571	11	2.201
6	2.447	12	2.179

Intervalos de confianza para $\bar{d}_{real}$

Acabamos de ver que si las diferencias entre los tiempos de ejecución de ambas máquinas se debieran a factores aleatorios, existiría un 95% de probabilidad de que t_{exp} se encuentre en el rango $[-t_{\frac{\alpha}{2}, n-1}, t_{\frac{\alpha}{2}, n-1}] = [-2.57, 2.57]$.

Como

$$t_{exp} = \frac{\bar{d} - \bar{d}_{real}}{s/\sqrt{n}} \in [-t_{\frac{\alpha}{2}, n-1}, t_{\frac{\alpha}{2}, n-1}]$$

sin más que identificar t_{exp} con los valores límite $\pm t_{\frac{\alpha}{2}, n-1}$ sabemos que habrá un 95% de probabilidad de que el valor medio real $\bar{d}_{real}$ de las diferencias entre los tiempos de ejecución se encuentre en el intervalo:

$$\bar{d}_{real} \in \left[\bar{d} - \frac{s}{\sqrt{n}} \times t_{\frac{\alpha}{2}, n-1}, \bar{d} + \frac{s}{\sqrt{n}} \times t_{\frac{\alpha}{2}, n-1} \right] = 24.3 \pm 20.9 = [3.4, 45.2] \text{ s}$$

Y el problema se transforma simplemente en comprobar si ese valor medio real $\bar{d}_{real}$ puede o no ser **cero**.

➡ En nuestro ejemplo, como el intervalo no incluye el cero, concluiremos que la hipótesis de que ambas máquinas pueden ser iguales **no** es cierta al 95% de confianza.

En resumen: Test t (valor-p o p-value)

Ejecución de n programas en dos máquinas A y B.

¿Son significativas las diferencias obtenidas ($d_i = tA_i - tB_i$)? Hay que usar mecanismos estadísticos.

Calculo: $t_{exp} = \frac{\bar{d}}{s/\sqrt{n}}$ siendo $\bar{d} = \frac{\sum_{i=1}^n d_i}{n}$ $s = \sqrt{\frac{\sum_{i=1}^n (d_i - \bar{d})^2}{n-1}}$

$$P - value = P(|t| \geq t_{exp}; n - 1)$$

Concluiremos, para un nivel de confianza del $(1-\alpha) \times 100$ % (típ. 95%) o para un nivel de significatividad de α (típ. 5%):

Si $P\text{-value} \geq \alpha$, entonces no hay diferencias significativas (es posible que los valores de d_i sean aleatorios $\rightarrow$ las dos alternativas pueden tener rendimientos equivalentes).

Si $P\text{-value} < \alpha$, entonces las alternativas presentan rendimientos significativamente diferentes. La que sea mejor dependerá del índice de rendimiento que se considere (tiempos medios, SPEC, etc.)

Resumen: Test t (Intervalos de confianza para t_{exp})

Ejecución de n programas en dos máquinas A y B.

¿Son significativas las diferencias obtenidas ($d_i = tA_i - tB_i$)? Hay que usar mecanismos estadísticos.

Calculo:
$$t_{exp} = \frac{\bar{d}}{s/\sqrt{n}} \quad \text{siendo } \bar{d} = \frac{\sum_{i=1}^n d_i}{n} \quad s = \sqrt{\frac{\sum_{i=1}^n (d_i - \bar{d})^2}{n-1}}$$

Intervalo de confianza para t_{exp} (para un nivel de significatividad α predeterminado, típ. 0.05):
$$\left[-t_{\alpha/2, n-1}, t_{\alpha/2, n-1} \right]$$

siendo $t_{\alpha/2, n-1}$ el valor que hace que $Prob(t \leq -t_{\alpha/2, n-1}) = \alpha/2$ para una distribución t de Student con $n-1$ grados de libertad.

Concluiremos, para un nivel de confianza del $(1-\alpha) \times 100$ % (típ. 95%) o para un nivel de significatividad α (típ. 5%):

Si t_{exp} está en el intervalo, entonces no hay diferencias significativas.

Si no lo está, entonces las alternativas presentan rendimientos significativamente diferentes.

Resumen: Test t (Intervalos de confianza para $\bar{d}_{real}$)

Ejecución de n programas en dos máquinas A y B.

¿Son significativas las diferencias obtenidas ($d_i = tA_i - tB_i$)? Hay que usar mecanismos estadísticos.

Intervalo de confianza para la media real de las diferencias $\bar{d}_{real}$ (para un nivel de significatividad α predeterminado, típ. 0.05):

$$\bar{d}_{real} \in \left[\bar{d} - \frac{s}{\sqrt{n}} \times t_{\frac{\alpha}{2}, n-1}, \bar{d} + \frac{s}{\sqrt{n}} \times t_{\frac{\alpha}{2}, n-1} \right] \equiv \bar{d} \pm \frac{s}{\sqrt{n}} \times t_{\frac{\alpha}{2}, n-1}$$

siendo $t_{\alpha/2, n-1}$ el valor que hace que $Prob(t \leq -t_{\alpha/2, n-1}) = \alpha/2$ para una distribución t de Student con $n-1$ grados de libertad.

Concluiremos, para un nivel de confianza del $(1-\alpha) \times 100$ % (típ. 95%) o para un nivel de significatividad α (típ. 5%):

Si el intervalo incluye el cero, entonces no hay diferencias significativas.

Si no incluye el cero, entonces las alternativas presentan rendimientos significativamente diferentes.

Ejemplo

Tiempos de ejecución (en segundos) de un benchmark compuesto por 5 programas (P1...P5) en dos máquinas diferentes (A y B)

Programa	tA (s)	tB (s)	$d_i = tA_i - tB_i$
P1	23	15	8
P2	28	22	6
P3	19	20	-1
P4	29	27	2
P5	36	39	-3
Suma	135	123	

¿Son significativas estas diferencias?

dato: $|t_{0.025,4}| = 2.78$

$$\bar{d} = 2.4s$$

$$s = 4.6s$$

$$t_{exp} = \frac{\bar{d}}{s/\sqrt{n}} = 1.16$$

$$P - value = P(|t| \geq t_{exp}; n - 1) = P(|t| \geq 1.16; 4) = 0.31(> @.@A)$$

Para un nivel de significatividad de $\alpha=0.05$:

Intervalo de confianza para t_{exp} : $[-2.78, 2.78]$ (**dentro del intervalo**)

Intervalo para $\bar{d}_{real}$: (**incluye el cero**)

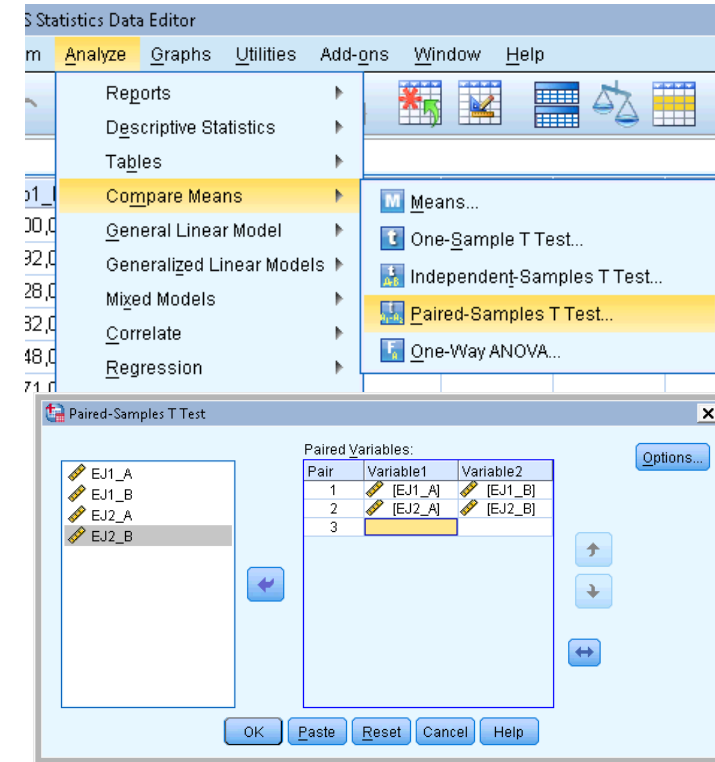
$$\bar{d} \pm \frac{s}{\sqrt{n}} \times t_{\frac{\alpha}{2}, n-1} = 2.4 \pm \frac{4.6}{\sqrt{5}} \times 2.78 = 2.4 \pm 5.72 = [-3.3, 8.1]s$$

NO podemos descartar, al 95% de nivel de confianza, que ambas máquinas puedan ser iguales.

Uso de SPSS para los intervalos de confianza

pairedtests.sav [DataSet1] - IBM SPSS Statistics Data Editor

	EJ1_A	EJ1_B	EJ2_A	EJ2_B
1	142,00	100,00	5,40	19,10
2	139,00	92,00	16,60	3,50
3	152,00	128,00	,60	3,40
4	112,00	82,00	1,40	2,50
5	156,00	148,00	,60	3,60
6	166,00	171,00	7,30	1,70
7				



Paired Samples Test									
		Paired Differences					t_{exp} ↓ t	df	P-value ↓ Sig. (2-tailed)
		Mean	Std. Deviation	Std. Error Mean	95% Confidence Interval of the Difference				
					Lower	Upper			
Pair 1	EJ1_A - EJ1_B	24,3333	19,9265	8,13497	3,42172	45,2449	2,991	5	,030
Pair 2	EJ2_A - EJ2_B	-,31667	9,03447	3,68831	-9,7978	9,16443	-,086	5	,935

Ejemplo Problema. Planteamiento

- Un equipo de programadores está diseñando una aplicación que trabaja fundamentalmente con vectores y matrices. Los programadores han propuesto dos algoritmos distintos, A y B, para su implementación, pero no tiene claro cuál de ellos utilizar y han llevado a cabo una serie de pruebas para comparar el rendimiento de ambos. Bajo una misma máquina han ejecutado varias ejecuciones con diferentes datos de entrada. El tiempo de respuesta en segundos se muestra en la siguiente tabla.

Algoritmo A	Algoritmo B
125	132
140	139
133	127
128	131
138	141
126	123
136	135

¿Cuál de los dos algoritmos ofrece mejores prestaciones?

¿Hay diferencias significativas en el rendimiento?

Ejemplo Problema. Solución

Para comparar el rendimiento de los dos algoritmos primero calculamos la suma de los tiempos de ejecución.

El algoritmo A tarda un total de 926 segundos, mientras que el B tarda 928 segundos. Así pues, ambos ofrecen un tiempo total de ejecución muy similar, presentando el algoritmo A una mejora de $928/926 = 1,0022$ sobre el B (aprox. un 1%).

Adicionalmente, podemos calcular un intervalo de confianza para las diferencias en los tiempos de ejecución y determinar si éstas son significativas. Como hay menos de 30 medidas utilizamos la distribución t-student. El intervalo de confianza para un nivel de confianza del 95% es:

$$\bar{d} \pm \frac{s}{\sqrt{n}} t_{\alpha/2, n-1} \text{ con } \alpha = 0.05$$

Ejemplo Problema. Solución

En particular, las diferencias observadas son:

Algoritmo A	Algoritmo B	Diferencia
125	132	-7
140	139	1
133	127	6
128	131	-3
138	141	-3
126	123	3
136	135	1

$$\bar{d} = \frac{\sum_{i=1}^n d_i}{n} \quad s = \sqrt{\frac{\sum_{i=1}^n (d_i - \bar{d})^2}{n-1}}$$

Su valor medio $\bar{d}$ es **-0.29** y la desviación típica s es **4.35**

$$\begin{aligned} \bar{d} \pm \frac{s}{\sqrt{n}} t_{\alpha/2, n-1} &= \\ &= -0.29 \pm \frac{4.35}{\sqrt{7}} \times 2.447 = \\ &= -0.29 \pm 4.02 \end{aligned}$$

El intervalo de confianza para la media de diferencias es $[-4.31, 3.74]$ y, dado que incluye el **0**, podemos concluir que no hay diferencias significativas al 95% de confianza en el rendimiento de los dos algoritmos. Por tanto, el equipo de programadores puede utilizar cualquiera de los dos algoritmos.

Intervalos de confianza de medias experimentales

Hipótesis: Realizamos n medidas d_i de un mismo fenómeno (p.ej. tiempos de ejecución de un programa, temperaturas CPU, tiempos acceso disco duro, productividades red,...). Estas pueden diferir debido a efectos aleatorios. Supondremos que $\{d_i\}$ se distribuye como una distribución normal de media $\bar{d}_{real}$. En ese caso, sabemos que

$$t_{exp} = \frac{\bar{d} - \bar{d}_{real}}{s/\sqrt{n}}$$

se distribuye según la distribución t-Student con $n-1$ grados de libertad. siendo $\bar{d}$ y s la media y la desviación típica muestrales, respectivamente.

Por tanto, hay un $(1-\alpha)*100\%$ de probabilidad de que el valor medio real $\bar{d}_{real}$ se encuentre en el intervalo:

$$\bar{d} \pm \frac{s}{\sqrt{n}} t_{\alpha/2, n-1}$$

Utilidad: Podemos usar esta información para saber si hace falta realizar más pruebas experimentales para determinar $\bar{d}_{real}$ con mejor precisión.

Queremos determinar un intervalo de confianza para el tiempo medio de escritura de un determinado fichero en un disco duro. Para ello, se han realizado varias medidas experimentales:

#exp	t _w (ms)
1	835
2	798
3	823
4	803
5	834
6	825
7	813
8	829

$$\overline{t_w} = \frac{\sum_{i=1}^n t_{wi}}{n} = 820ms \quad s = \sqrt{\frac{\sum_{i=1}^n (t_{wi} - \overline{t_w})^2}{n-1}} = 14ms$$

Por tanto, hay un 90% ($\alpha=0.1$) de probabilidad de que el tiempo medio de escritura real de ese fichero se encuentre en el intervalo:

$$820 \pm \frac{14}{\sqrt{8}} t_{0.1, \frac{2}{2}, 8-1} = [811, 829]ms$$

Igualmente, hay un 95% ($\alpha=0.05$) de probabilidad de que el tiempo medio de escritura real se encuentre en el intervalo:

$$820 \pm \frac{14}{\sqrt{8}} t_{0.05, \frac{2}{2}, 8-1} = [808, 832]ms$$